Aror University of Art, Architecture, Design & Heritage Sukkur.

Department of Artificial Intelligence

Data Structures (Fall-2026)

LAB No. 1

Objective of Lab No. 1:
After performing lab1, students will be able to:

- Install Anaconda
- Understand the basic terminology of Data Structures
- Configure Anaconda – Jupytor Notebook Environment
- Create new Program or Project
- Write first program in Python
- Understand the basics of program
- Understand the basic concepts of Python data structures.
- Use Python lists, tuples, sets, and dictionaries for data organization.
- Implement basic operations associated with Abstract Data Types (ADTs).
- Understand the relationship between input size and algorithm performance.
- Analyze simple algorithms using Big-O notation.
- Compare the time complexity of common Python operations and algorithms.

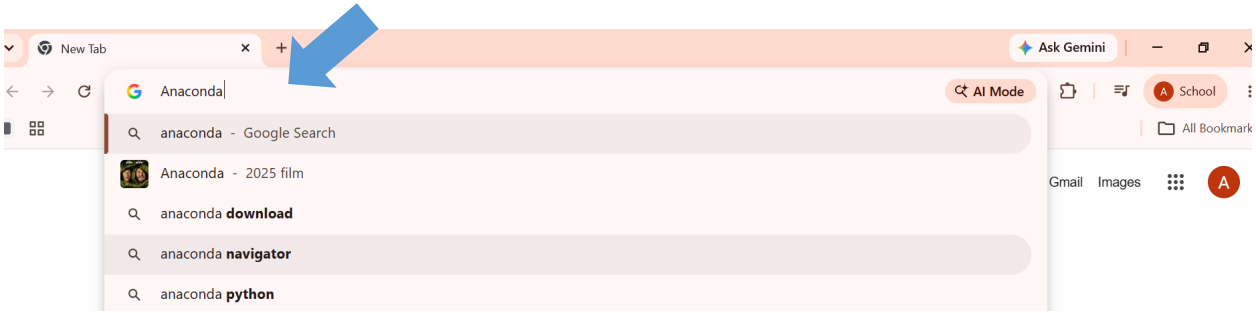
Installing Anaconda:

DevC++ is a free integrated development environment for Windows operating system which is distributed under GNU general public license for programming in C and C++. An integrated development environment provides comprehensive facilities to programmers. It normally consists of source code editor, build automation tools and debugger. Most modern IDEs offer intelligent code completion features. Follow the below given steps to install the DevC++.

1. To install Anaconda, you first need Anaconda setup. You can download the Anaconda setup from the official website i.e. anaconda.com. Link is shared here: https://www.anaconda.com/download

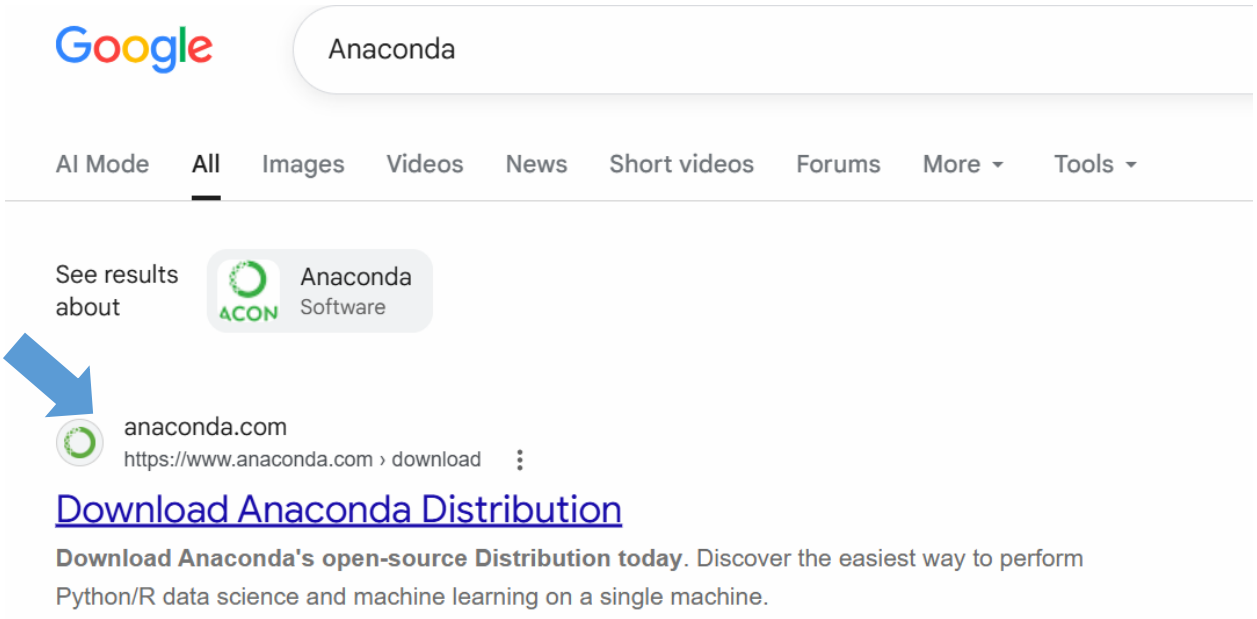
2. Open google.com and type anaconda.

Aror University of Art, Architecture, Design & Heritage Sukkur.



3. Click on the first link

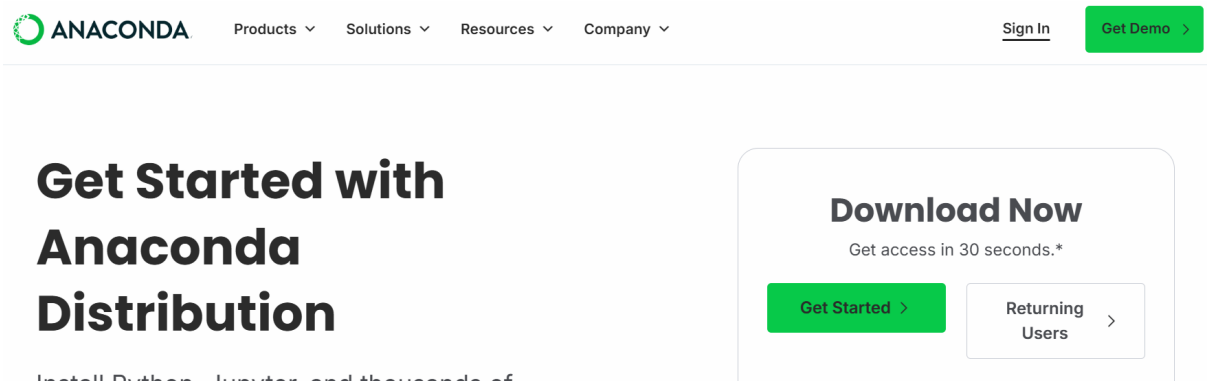
4.



4. In the next step, Sign In first with your google account:



Aror University of Art, Architecture, Design & Heritage Sukkur.



Aror University of Art, Architecture, Design & Heritage Sukkur.

Email

Forgot Password?

By clicking Sign In with Email, or signing in with Google, Microsoft, or GitHub, I agree to the collection of my data, Anaconda's Privacy Policy and Terms of Service. These terms do not supersede any existing Master Subscription Agreement or other enterprise agreement entered into with Anaconda.

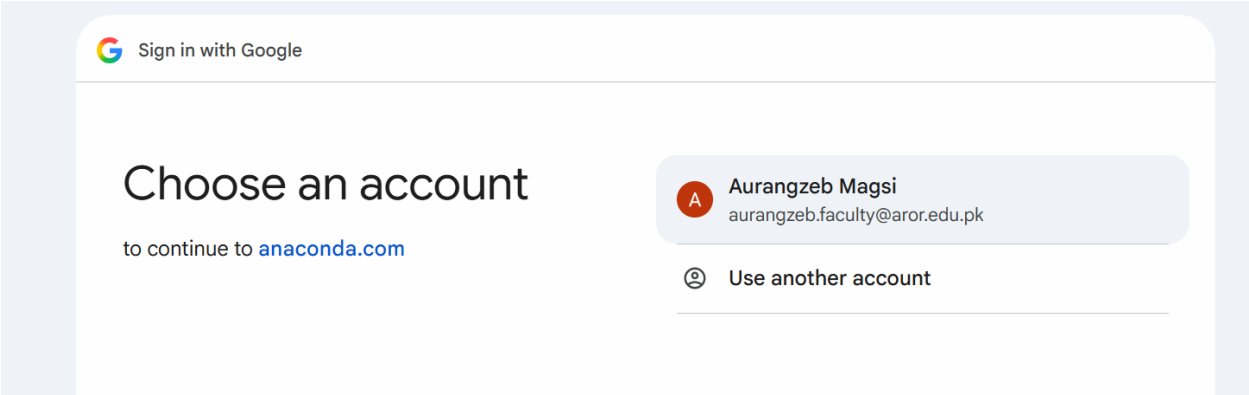
Sign In with Email

or sign in with

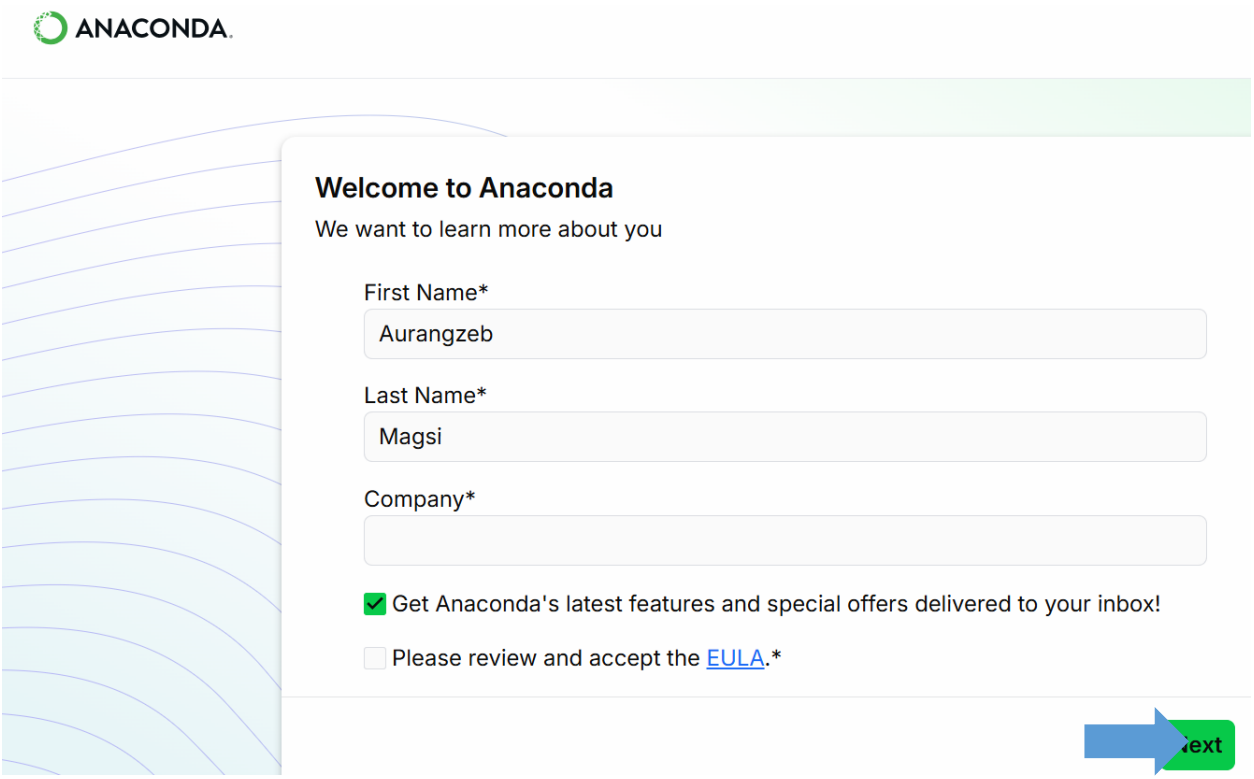
Google Microsoft GitHub

Don't have an account? Sign Up

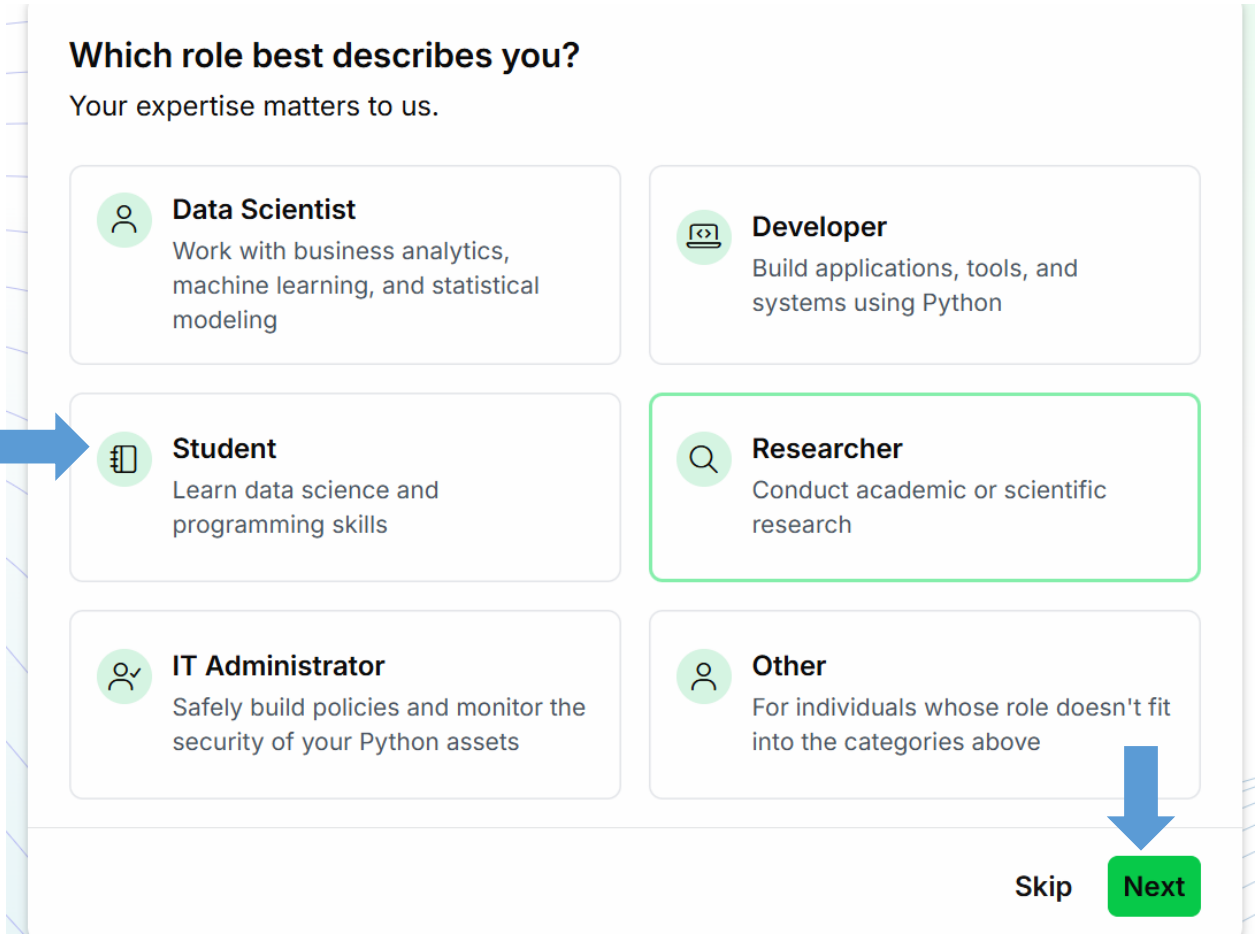




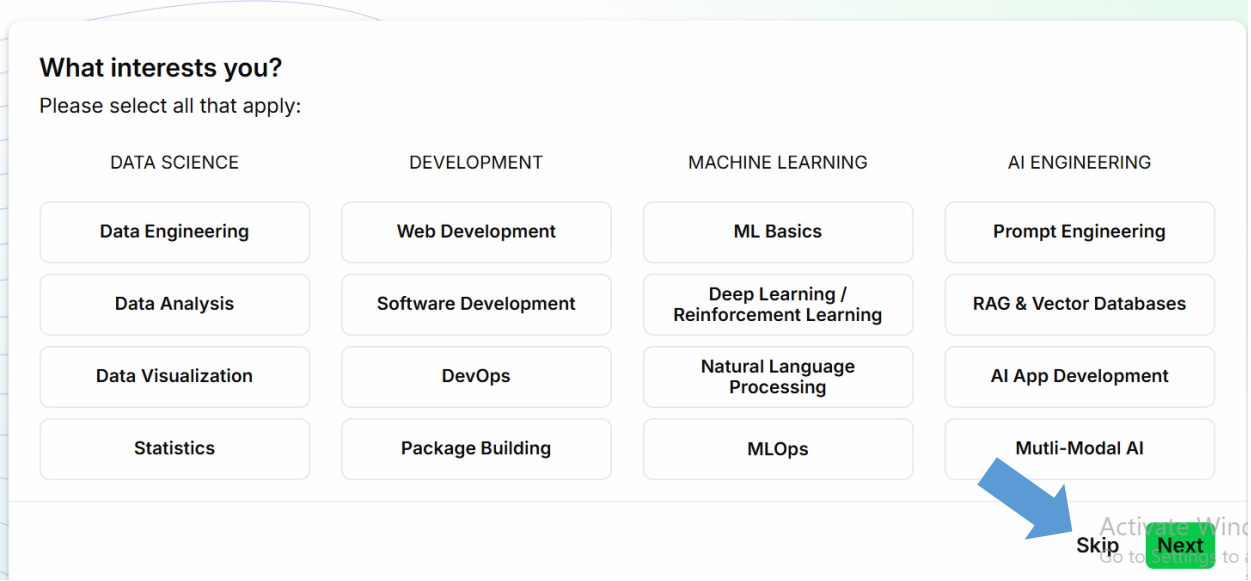
5. Click on ‘Next’ button



6. Select any appropriate option and click on ‘Next’ button

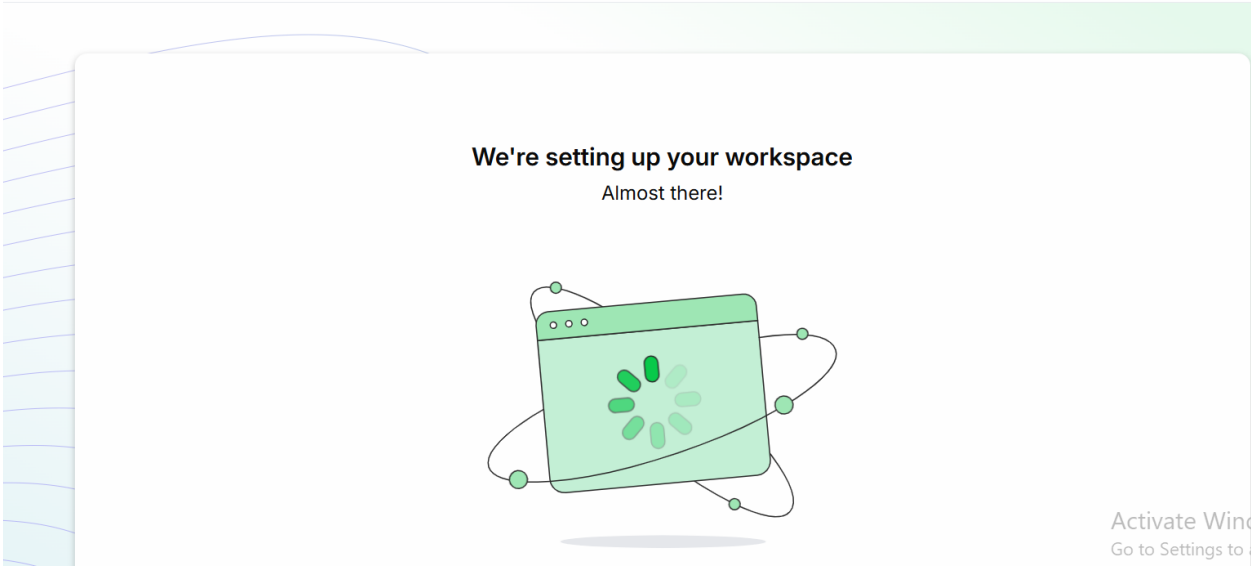


7. Click on ‘Skip’ button

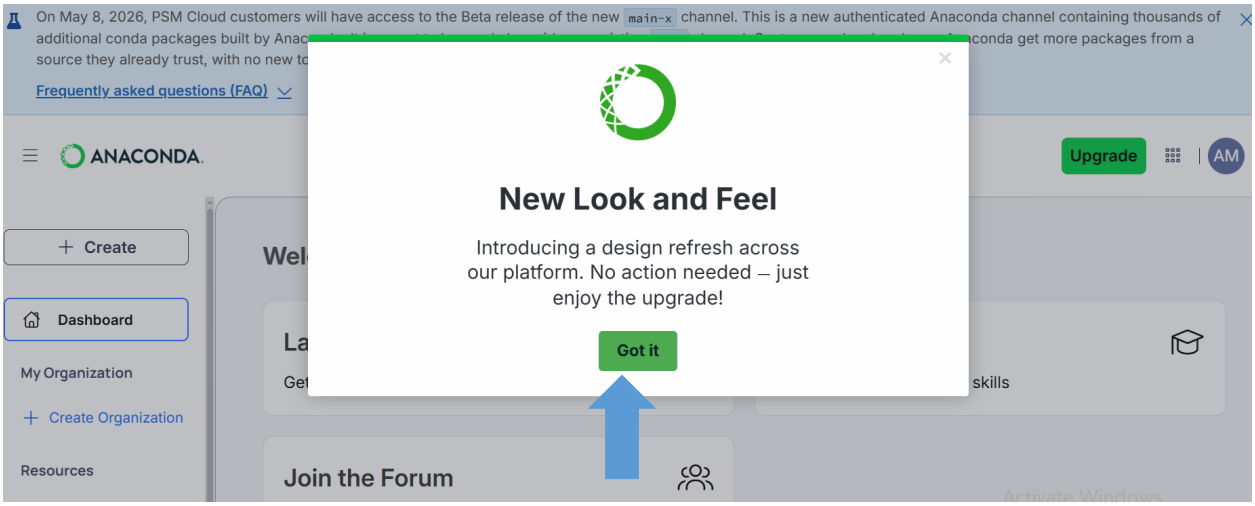


8. Wait untill setup completed

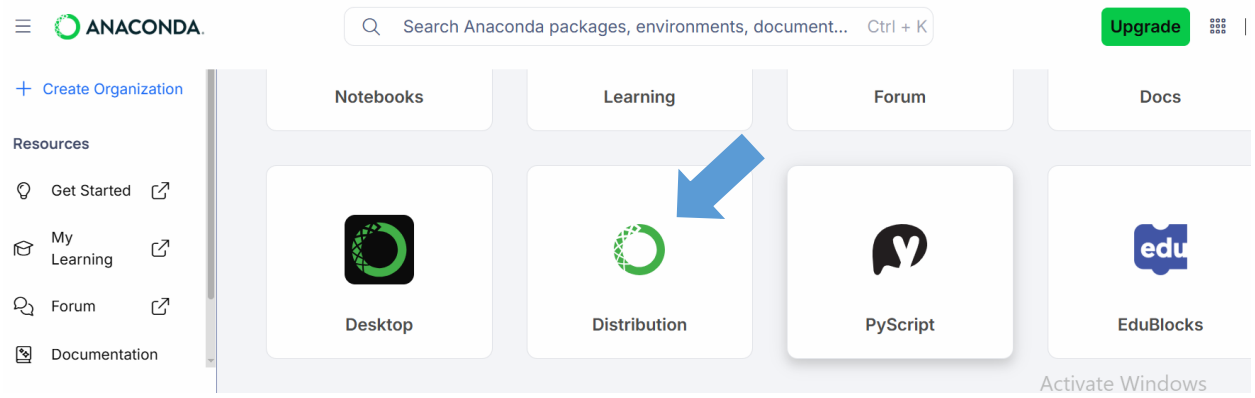
ANACONDA.



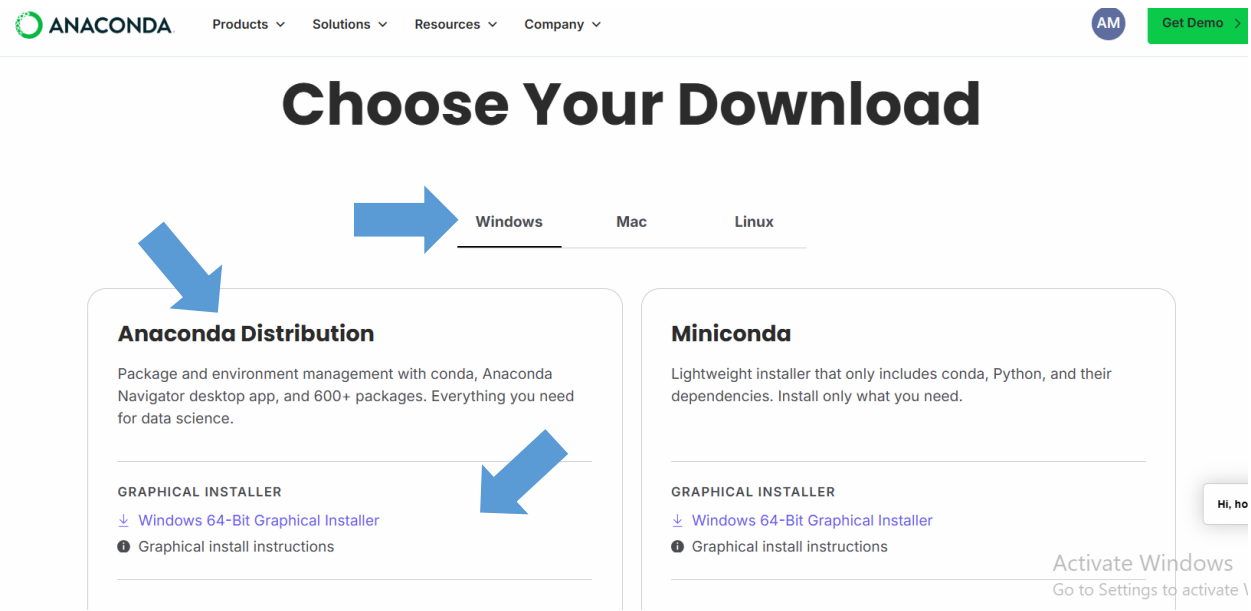
9. After all settings, click on ‘Got it’ button



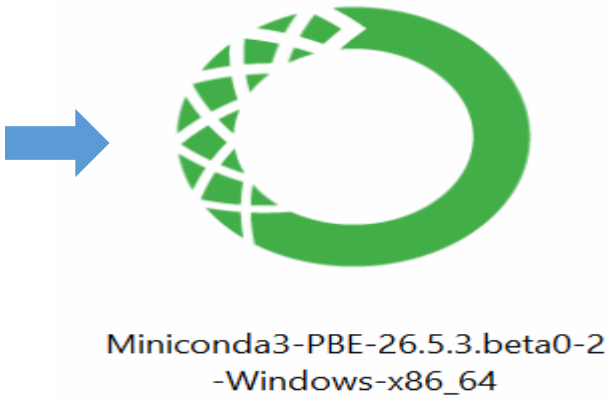
10. Scroll down and go with the Distribution option:



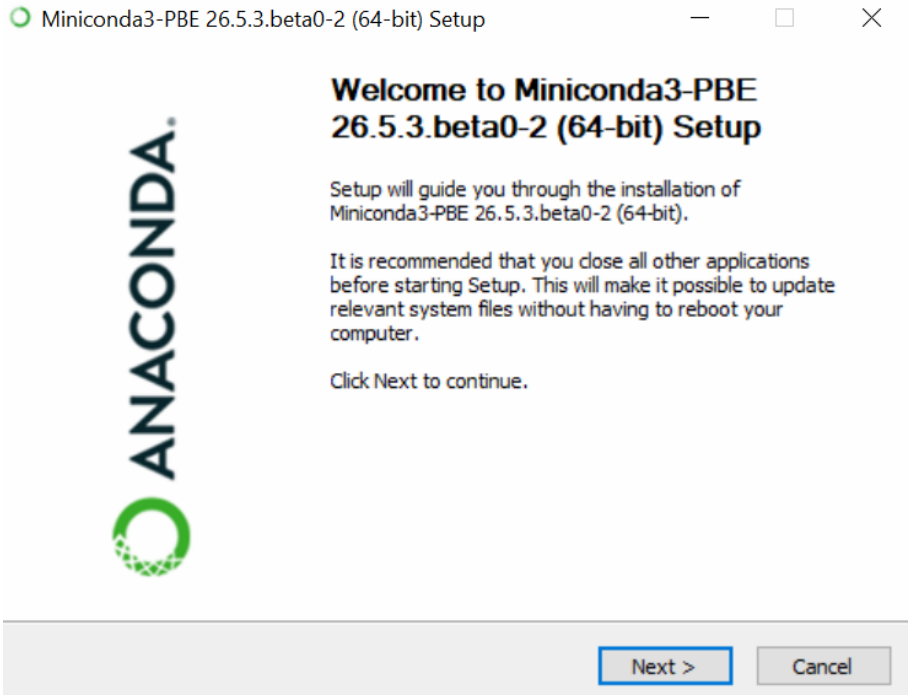
11. In the next page, go with the ‘Wiondows’ option, and download the ‘Anaconda Distribution’ file:



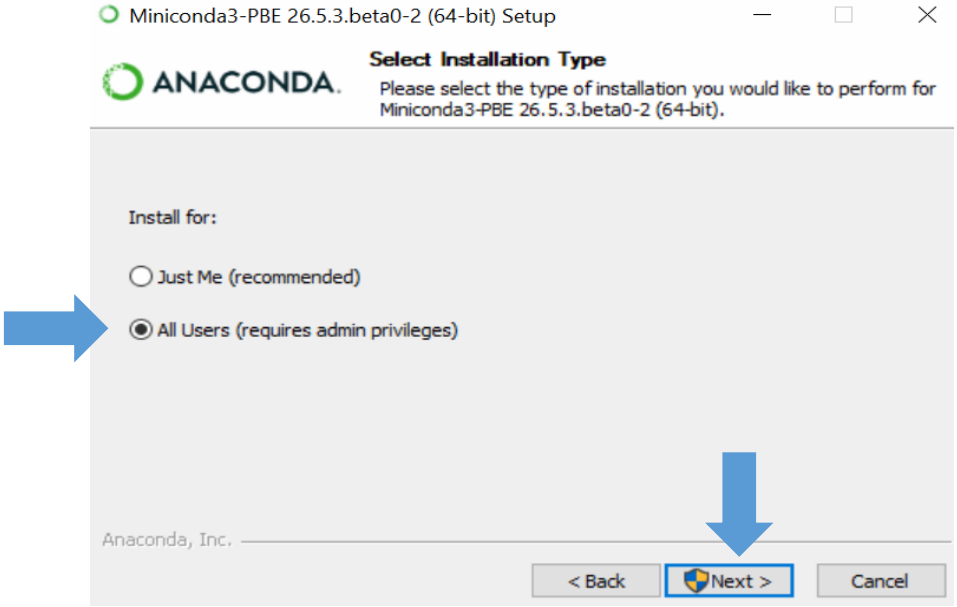
12. Double click on the downloaded file and start installation:



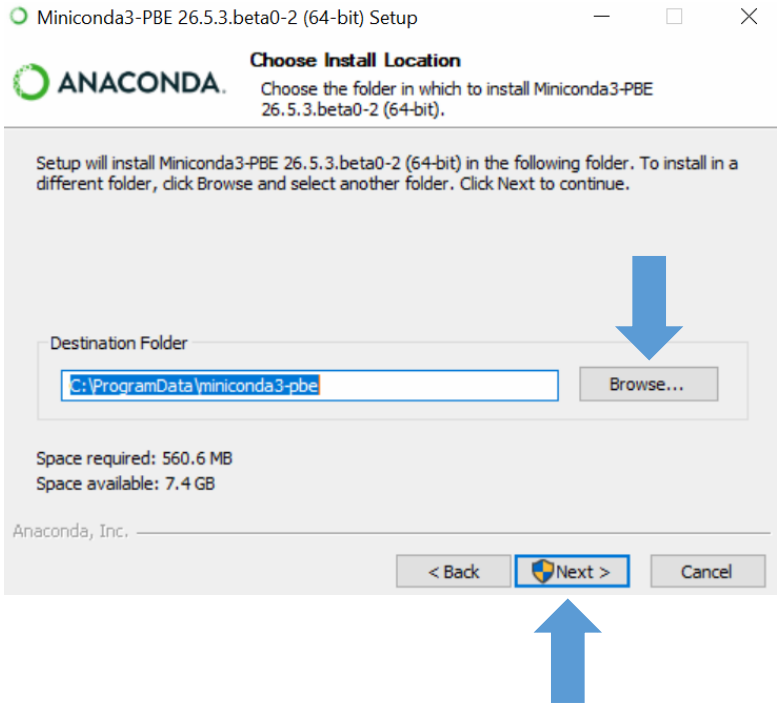
13. Click on ‘next’ button



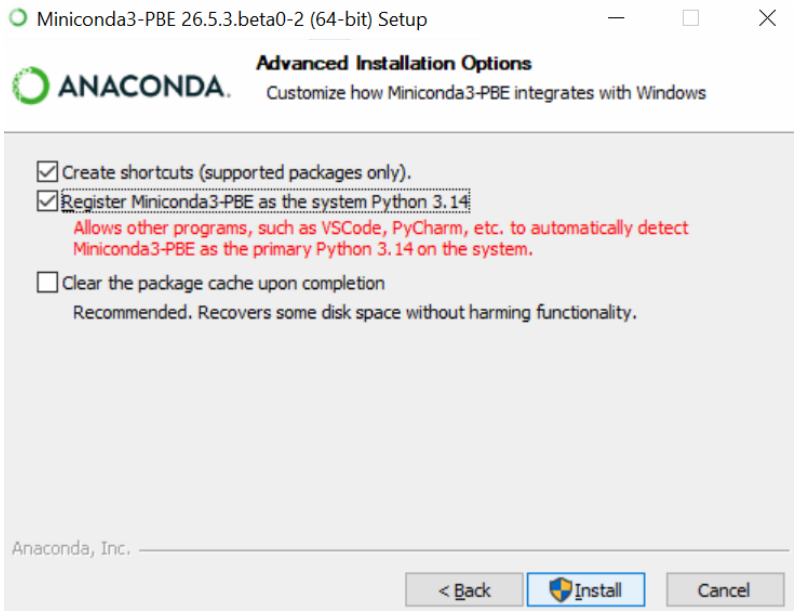
14. Click on ‘I Agree’ then check the ‘All Users’ button then proceed with ‘Next’ button



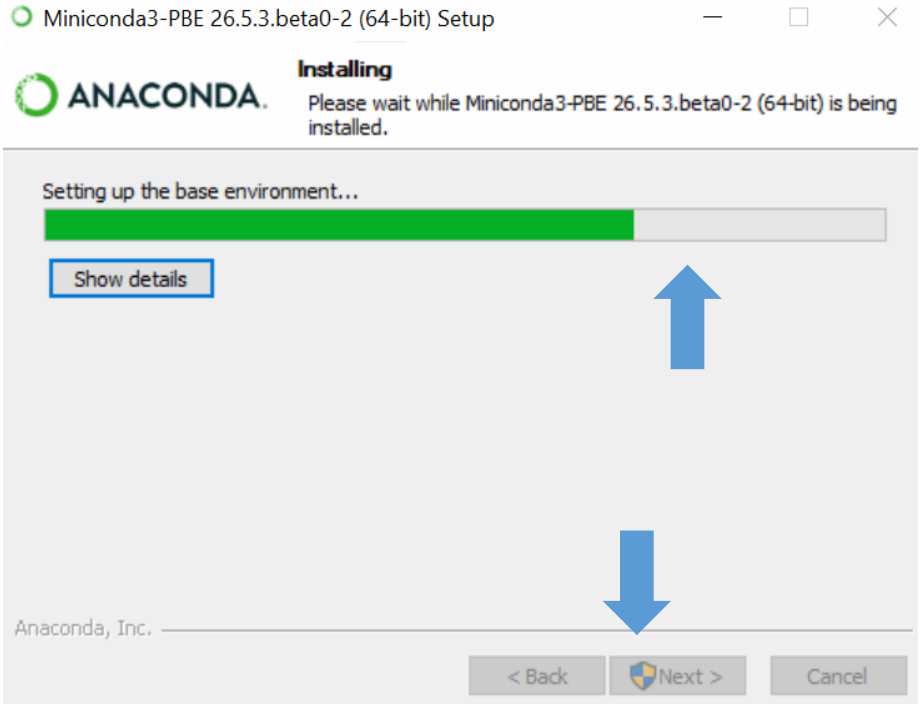
15. Set the desired location and click ‘Next’ button

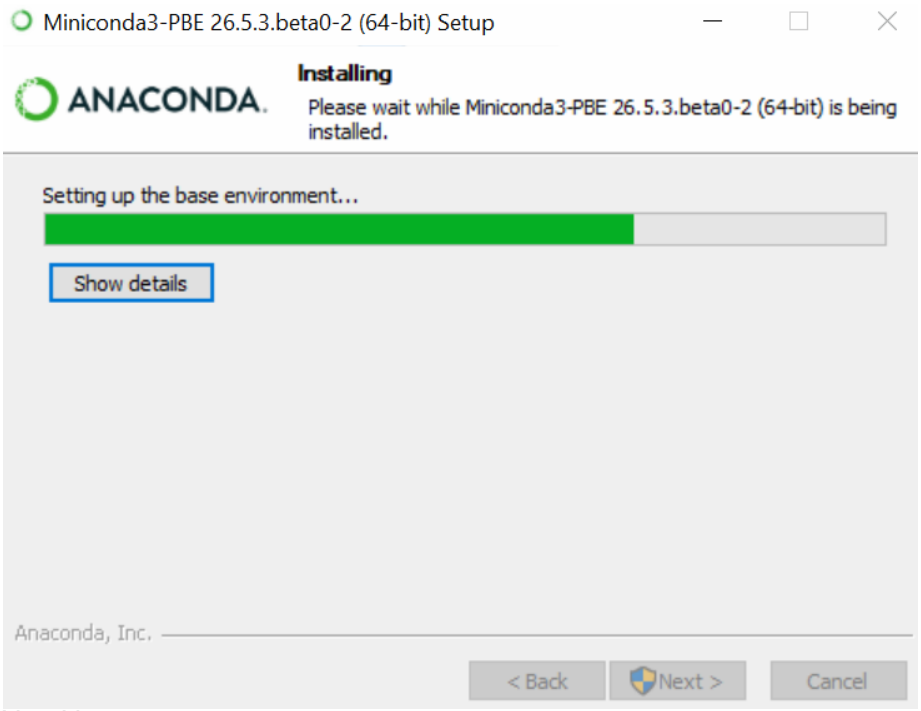


16. Check the ‘Registration’ option and click ‘Install’ button

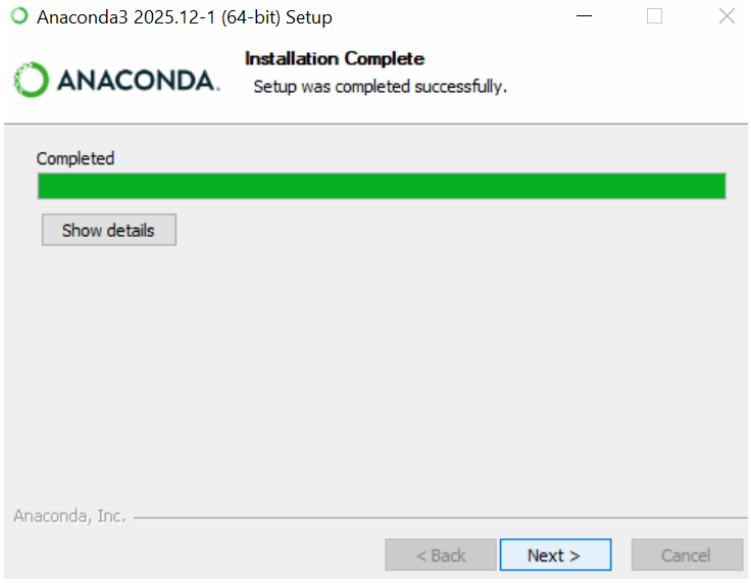


17. Wait until installation completes. Once setup complete, click on ‘Next’ button and then click on ‘Finish’ button.

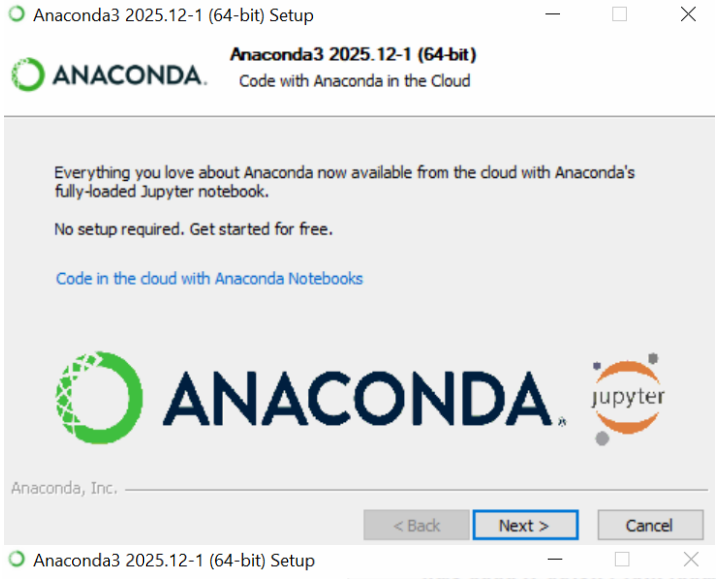


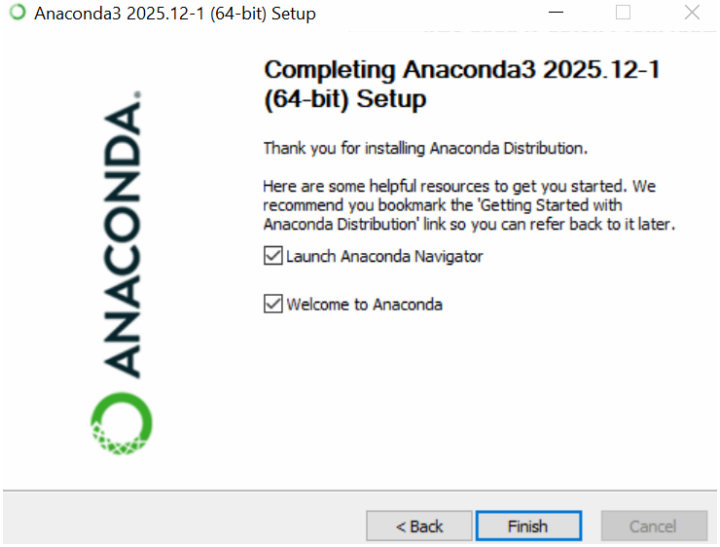


18. Click on ‘Next’ button



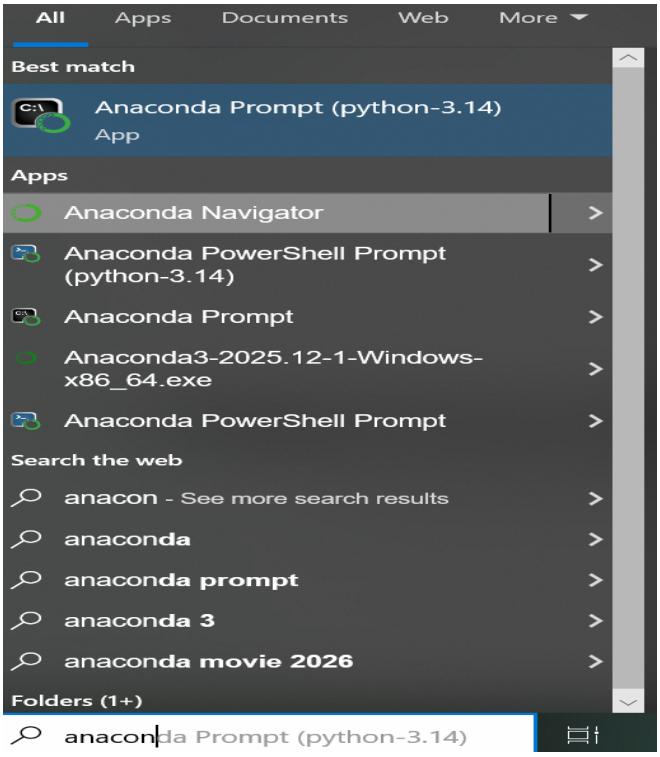
19. Click on ‘Next’ button and then ‘Finish’ button.



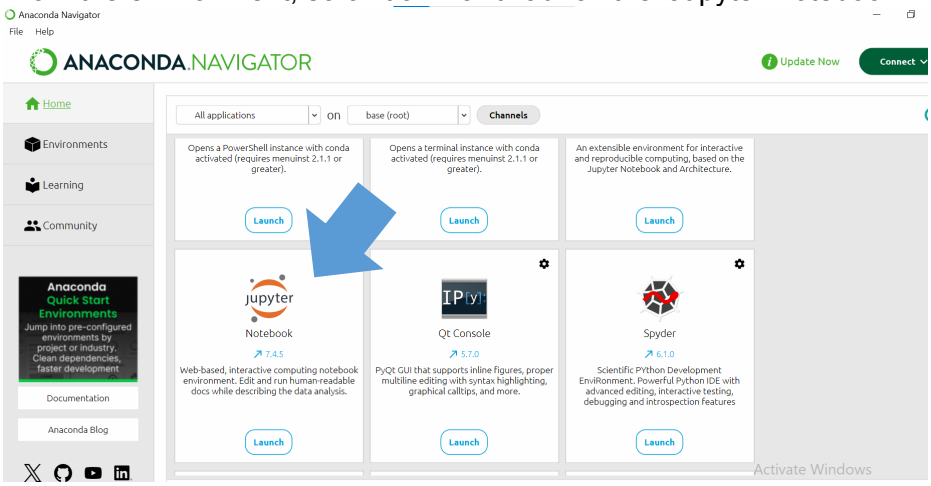


20. Go to Start menu, type ‘Anaconda’ and select ‘Anaconda Navigator’





21. From the environment, scroll down and launch the 'Jupyter Notebook'



Introduction to Python Data Structures:

Data structures are organized ways of storing and managing data so that it can be accessed and

processed efficiently. Python provides several built-in data structures such as lists, tuples, sets, and dictionaries. These structures are frequently used as building blocks for implementing more advanced data structures and algorithms.

An Abstract Data Type (ADT) defines the data and the operations that can be performed on it without specifying how the data is internally implemented. For example, a Stack ADT may provide push, pop, and peek operations. In this lab, students will use Python data structures to understand basic ADT operations and algorithm efficiency.

Python Built-in Data Structures:

Data Structure	Description	Example
List	Ordered and mutable collection.	[10, 20, 30]
Tuple	Ordered and immutable collection.	(10, 20, 30)
Set	Unordered collection of unique elements.	{10, 20, 30}
Dictionary	Collection of key-value pairs.	{"Name": "Ali", "Age": 20}

Example Program:

```
# Python Data Structures Example

students = ["Ali", "Sara", "Ahmed"]
courses = ("DSA", "Python", "AI")
departments = {"AI", "MMG", "CYS"}
student = {"Name": "Ali", "Semester": 3}
print("List:", students)
print("Tuple:", courses)
print("Set:", departments)
print("Dictionary:", student)

# Basic list operations
students.append("Hina")
students.remove("Sara")

print("Updated List:", students)
print("Number of students:", len(students))
```

Understanding the Program:

students = [...]: Creates a Python list that stores multiple student names.
courses = (...): Creates a tuple. Unlike a list, a tuple cannot normally be modified after creation.
departments = {...}: Creates a set that stores unique values.
student = {"Name": "Ali", "Semester": 3}: Creates a dictionary containing key-value pairs.
append(): Adds a new element to the end of a list.
remove(): Removes a specified element from a list.
len(): Returns the number of elements in a collection.

1. List

A **list** is an ordered and mutable collection. It is useful when we need to store multiple items and may need to **add, remove, or modify** them.

Example:
['Ali', 'Ayesha', 'Ahmed', 'Hina']

2. Tuple

A **tuple** is an ordered but **immutable** collection. Once created, its elements cannot normally be changed.

Example:
```
courses = ("Python", "Data Structures", "AI")
print(courses[1])
```

3. Set

A **set** is a collection of **unique elements**. It automatically removes duplicate values and does not support indexing.

Example:
```
numbers = {10, 20, 30, 20, 10}
print(numbers)
```

4. Dictionary

A **dictionary** stores data as **key-value pairs**. Each key must be unique.

Example:
```
student = {
   "name": "Ali",
   "sap_id": "AI-101",
   "semester": 3
}
print(student["name"])
print(student["semester"])
```

Basic Abstract Data Type (ADT) Operations:
An ADT specifies what operations are available on a data structure. The following operations will be practiced using Python collections:

- Create: initialize an empty or populated data structure.

- Insert: add an element to the data structure.
- Delete: remove an element.
- Search: determine whether an element exists.
- Access: retrieve an element or value.
- Traverse: process each element of the structure.

Introduction to Big-O Notation:
Big-O notation describes how the running time or memory requirement of an algorithm grows as the input size increases. Common complexities include O(1), O(log n), O(n), O(n log n), and O(n²).

```
# O(1)
def get_first(data):
    return data[0]
# O(n)
def display_all(data):
    for item in data:
        print(item)
# O(n^2)
def compare_all(data):
    for i in data:
        for j in data:
            print(i, j)
```

The first function performs a constant number of operations, so its complexity is O(1). The second processes each element once, giving O(n). The third contains a nested loop that processes pairs of elements, giving O(n²).

Lab Tasks:

Task 01: Create a Python program that stores the names of five students in a list. Add two new names, remove one name, and display the updated list.

Task 02: Create a tuple containing five course names. Display the first, last, and middle elements and explain why the tuple is immutable.

Task 03: Create a set containing repeated values. Display the set and demonstrate how duplicate values are automatically removed. Perform union and intersection operations on two sets.

Task 04: Create a dictionary containing a student's Name, SAP ID, Program, and Semester. Add a new key-value pair, update an existing value, delete a key, and display all keys and values.

Task 05: Implement a simple ADT-style program for a collection that supports Create, Insert, Delete, Search, and Traverse operations.

Task 06: Write three functions demonstrating O(1), O(n), and O(n²) time complexity. Test each function with different input sizes and record the observed execution behavior.

Task 07: For each of the following operations, identify the expected time complexity and provide a brief justification: list indexing, searching an unsorted list, dictionary key lookup, and traversing a list.

Task 08: Write a short comparison explaining when a list, tuple, set, or dictionary should be preferred for storing data.

Submission Requirements:

- Save the Python program/notebook using the format: SAPID_YourName_Lab01.
- Write your SAP ID and Name at the beginning of the notebook.



Aror University of Art, Architecture, Design & Heritage Sukkur.

- Run all programs and include the output for each task.
- Add brief comments to explain important parts of the code.
- Submit the completed lab on the google classroom.